SpiralTrain
developer courses



Python Libraries and Tools

Optional Topics

Choosing Topics

- The optional modules are selected with the trainer to suit the group.
- Command-line programs and subprocesses support scripting and automation.
- The modern toolbelt covers progress displays, formatted output and HTTP.
- Web scraping and SQLAlchemy cover collecting and storing records.
- NumPy and pandas cover arrays, tables and reports.
- GUI toolkits introduce desktop windows.
- Inheritance previews Advanced Python and can be skipped.
- The supplied data and database seed let later modules run independently.

SpiralTrain developer courses

- ▶ **Inheritance**
- Command-line Programs
- Modern Toolbelt
- Web Scraping
- SQLAlchemy
- NumPy
- Pandas
- GUI Toolkits

APPENDIX

- Running Other Programs

- Inheritance
- Duplicated Code
- Cost of Duplication
- Base Classes and Subclasses
- Inherited Members
- Constructor Chaining
- Overriding Methods
- Extending with super
- isinstance and issubclass
- str and repr
- Polymorphism
- Abstract Base Classes
- Enforcing the Contract
- Composition
- Scraper Toolkit
- Advanced Python Topics

SpiralTrain developer courses

Inheritance

- **A first course builds one class at a time :**
  - A blueprint, its attributes, its methods, and `self`
- **Working code is rarely a single class :**
  - It is several classes doing one job on different data
- **This module is about the relationship between them :**
  - What they share, what each one changes, where to draw the line
- **The worked example is a scraper :**
  - One base class, two sites, and nothing written twice
- **The same design comes back in the web scraping module**

SpiralTrain developer courses

Duplicated Code

- **A scraper for one site is a small class :**
  - Collect the rows, keep them, report what was found
- **A second site needs that class again :**
  - Only the parsing differs, because only the markup differs
- **Copying the file is the fastest thing to do :**

```
class BooksScraper:
    def __init__(self, name: str) -> None: ...
    def collect(self) -> None: ...
    def parse(self) -> list[dict[str, str]]: ...   # the only site-specific part
    def report(self) -> str: ...

class QuotesScraper:            # the same file again, one method changed
    ...
```

- **Four of the five methods end up identical, character for character**

demo01
duplication

SpiralTrain developer courses

Cost of Duplication

- **A bug fixed in one copy stays alive in the other :**
  - Nothing in the code records that the two files belong together
- **A new field means an edit in every copy :**
  - The number of copies only ever grows
- **A reader cannot tell the shared part from the deliberate difference :**
  - Both look the same on the page
- **The repair is to write the shared part once :**
  - And let each site supply only what is genuinely its own

SpiralTrain developer courses

Base Classes and Subclasses

- **A class may name another class as its base :**
  - Written as `class BooksScraper(Scraper):`
- **The base class holds everything that is not site-specific**
- **Each subclass supplies only the part that differs :**

```
class Scraper:
    def __init__(self, name: str) -> None:
        self.name: str = name
        self.rows: list[dict[str, str]] = []
    def collect(self) -> None:
        self.rows = self.parse()
    def parse(self) -> list[dict[str, str]]:
        return []

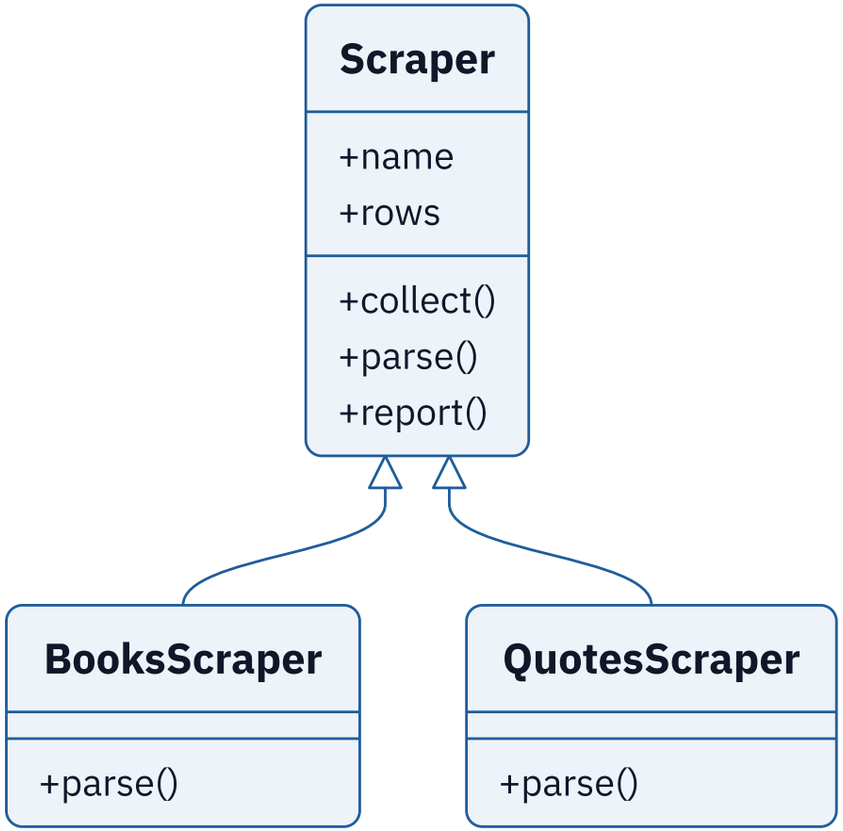
class BooksScraper(Scraper):
    def parse(self) -> list[dict[str, str]]:
        return [row.copy() for row in BOOK_ROWS]
```

demo02
subclass

SpiralTrain developer courses

Inherited Members

- Every method the base class defines, without repeating it
- Every attribute the base `__init__` creates, for the same reason
- A subclass may add to that, replace part of it, or leave it alone



SpiralTrain developer courses

Constructor Chaining

- **A subclass with attributes of its own needs its own `__init__` :**
  - The base initialiser still has to run
- **`super().__init__(...)` calls the version one level up :**
  - Call it first, before setting anything of your own

```
class BooksScraper(Scraper):
    def __init__(self, name: str, base_url: str) -> None:
        super().__init__(name)          # name and rows exist after this line
        self.base_url: str = base_url
```

- **Leaving that line out is the most common mistake in the module :**
  - The object is built happily, then fails later with `AttributeError`

demo03
super_init

SpiralTrain developer courses

Overriding Methods

- **A method on the subclass wins over the one on the base :**
  - Same name, different body, no declaration needed anywhere
- **Replacing a method throws the base version away :**

```
class Scraper:
    def clean(self, value: str) -> str:
        return value.strip()

class BooksScraper(Scraper):
    def clean(self, value: str) -> str:
        return value.removeprefix("$")      # the strip() is gone
```

- **Given the raw value `"  $51.77  "` that override returns it unchanged :**
  - The leading spaces are still there when `removeprefix` looks

SpiralTrain developer courses

Extending with super

- `super()` reaches the base version from inside the override :

```
class BooksScraper(Scraper):
    def clean(self, value: str) -> str:
        return super().clean(value).removeprefix("$")
```

- The base class does its part, the subclass adds one step
- Choose between the two deliberately :
  - Replace when the inherited behaviour is wrong here
  - Extend when it is right and merely incomplete
- Overriding `__init__` is the same decision, and usually the extending one

demo04
overriding

SpiralTrain developer courses

isinstance and issubclass

- **`isinstance(obj, Class)` asks whether an object is usable as that class :**
  - True for the class itself and for every subclass of it
- **`issubclass(Sub, Base)` asks the same question about two classes**
- **`type(obj) is Class` is a stricter and usually wrong question :**
  - It refuses subclasses, which defeats the point of having them

```
isinstance(paged, BooksScraper)     # True, PagedBooksScraper inherits from it
type(paged) is BooksScraper         # False, the type is not exactly that
isinstance(value, (str, bytes))     # a tuple asks about several at once
```

- **Reach for it only where the code genuinely has to branch on type**

SpiralTrain developer courses

str and repr

- **`__str__` is for the person reading the output :**
  - Used by `print()` and by `f"{value}"`
- **`__repr__` is for the person debugging :**
  - Used by the interpreter, and by every container
- **A list shows the `__repr__` of its elements, never the `__str__` :**
  - Which is why a list of objects prints hexadecimal addresses

```
class Scraper:
    def __str__(self) -> str:
        return f"{self.name} with {len(self.rows)} rows"
    def __repr__(self) -> str:
        return f"{type(self).__name__}(name={self.name!r}, rows={len(self.rows)})"
```

- **Define both once on the base class and every subclass inherits them**

demo06
str_and_repr

SpiralTrain developer courses

Polymorphism

- **One list may hold objects of several classes at once :**
  - Provided they answer to the same methods
- **The loop calls the method and Python picks the version :**

```
def run_all(scrapers: list[Scraper]) -> int:
    total = 0
    for scraper in scrapers:
        scraper.collect()            # a different parse() runs each time round
        print(scraper.report())
        total += len(scraper.rows)
    return total
```

- **Adding a fourth site changes the list, and nothing else**
- **This is polymorphism, and it is the reason the base class earns its place**

demo07
polymorphism

SpiralTrain developer courses

Abstract Base Classes

- A base `parse()` that returns an empty list is a promise nobody enforces :
  - A subclass that forgets it scrapes nothing, silently
- An abstract base class states the requirement and enforces it :

```
from abc import ABC, abstractmethod

class Scraper(ABC):
    @abstractmethod
    def parse(self) -> list[dict[str, str]]:
        """Return the rows found in one page of this site."""

    def collect(self) -> None:          # concrete, and calls the abstract method
        self.rows = self.parse()
```

- A subclass without `parse` cannot be created at all
- Neither can the abstract class itself, which is the point of it

demo08
abstract_contract

SpiralTrain developer courses

Enforcing the Contract

- **The failure moves to the earliest moment it possibly can :**

```
TypeError: Can't instantiate abstract class HalfFinishedScraper
without an implementation for abstract method 'parse'
```

- **The requirement is written down where the next reader will find it**
- **The base class may still carry real shared code :**
  - Abstract methods and ordinary methods sit side by side
- **Use one when several classes must offer the same set of methods :**
  - Leave it out for a base class that already works on its own

SpiralTrain developer courses

Composition

- **Inheritance is a strong claim, so say it out loud before writing it :**
  - A `BooksScraper` is a `Scraper`, so it inherits
  - A `BooksScraper` is not a `RowStore`, so it holds one instead
- **Holding another object as an attribute is composition :**

```
class BooksScraper(Scraper):
    def __init__(self, name: str, store: RowStore) -> None:
        super().__init__(name)
        self.store: RowStore = store
```

- **A wrong base class publishes methods nobody meant to offer :**
  - Inheriting from `dict` gives a scraper eleven public dictionary methods
- **Prefer composition where the choice is unclear, as it is easier to undo**

demo09
composition

SpiralTrain developer courses

Scraper Toolkit

- **The finished shape, in one file :**
  - An abstract `Scraper`, with `parse` as the contract
  - Shared `clean`, `collect`, `row_count`, `__str__` and `__repr__`
  - `BooksScraper` and `QuotesScraper`, one method each
- **The rows are canned, so nothing here touches the network**
- **The web scraping module replaces `parse` with real HTML :**
  - Every other line of the file stays exactly as it is

demo10
scraper_toolkit

SpiralTrain developer courses

Advanced Python Topics

- **The same topic, carried a long way further :**
  - Multiple inheritance, and the order Python searches the bases in
  - Mixins, small classes that add one behaviour to many others
  - `Protocol`, a contract a class satisfies without ever naming it
- **And the machinery underneath the classes :**
  - `__new__`, descriptors, metaclasses and `__slots__`
- **Everything above assumes exactly what this module has just covered**
- **Nothing on this slide is needed for the rest of this course**

SpiralTrain developer courses

Summary : Inheritance

- **Inheritance exists to stop the same code being written twice :**
  - Shared behaviour in a base class, differences in the subclasses
- **A subclass inherits methods and the attributes the base `__init__` creates :**
  - `super().__init__(...)` runs that initialiser, and is easy to forget
- **Overriding replaces a method, and `super()` inside it extends instead**
- **`__str__` is for output and `__repr__` is for debugging :**
  - Containers always show the second one
- **One loop over a list of related objects calls a different method each time**
- **An abstract base class turns the promise into a rule the interpreter enforces**
- **Composition, a plain attribute, is the answer whenever "is a" sounds wrong**

exercise01
Inheritance

- Inheritance
- **Command-line Programs**
- Modern Toolbelt
- Web Scraping
- SQLAlchemy
- NumPy
- Pandas
- GUI Toolkits

APPENDIX

- Running Other Programs

- Command-line Programs
- sys.argv
- Manual Argument Parsing
- ArgumentParser
- Options and Flags
- Mutually Exclusive Options
- Generated Help and Errors
- Subcommands
- Path Arguments
- FileType
- main and the Exit Status
- Standard Output and Standard Error
- Inline Script Metadata
- Executable Scripts
- Command-line Checklist

SpiralTrain developer courses

Command-line Programs

- **A script does one job, to one file, with the settings typed into the source**
- **A tool takes its settings from the command line :**
  - The same code serves every file, every level and every output format
- **Three things separate the two :**
  - It explains itself when asked
  - It refuses a wrong command line clearly, instead of crashing later
  - It reports success or failure in a way another program can read
- **Everything in this module is standard library, so it ships with Python**
- **The end of the module is a checklist you can hold a script up against**

SpiralTrain developer courses

sys.argv

- **Everything typed after the program name arrives in `sys.argv` :**
  - A plain list of strings, in the order they were given
  - `sys.argv[0]` is the program itself, so the real arguments start at `1`
- **Nothing is converted, so a number arrives as text**

```
import sys

print(sys.argv)          # ['tool.py', 'report', '--limit', '5']
print(sys.argv[1:])      # ['report', '--limit', '5']
```

- **This is the whole interface between the shell and the program**
- **Every argument library in Python is a layer over this one list**

demo01
sys_argv

SpiralTrain developer courses

Manual Argument Parsing

- **Reading the list yourself works, right up until it does not :**
  - An option whose value is missing walks off the end of the list
  - A value that should be a number is still a string
  - A mistyped option is silently taken for something else
  - Nobody wrote `--help`, so there is none
- **Each of those is a few more lines of defensive code**

```
if item == "--limit":
    index += 1
    limit = int(argv[index])   # IndexError, or ValueError
```

- **The handwritten version grows faster than the program it serves**

demo02
argv_by_hand

SpiralTrain developer courses

ArgumentParser

- `argparse` inverts the work : you describe the command line, it does the rest
- A positional argument is named and required by position

```
import argparse

parser = argparse.ArgumentParser(description="Print the square of a number.")
parser.add_argument("number", type=int, help="the number to square")
args = parser.parse_args()

print(args.number ** 2)
```

- `type=` is a conversion, so `args.number` is an `int` and not `"12"`
- `help=` is the text that appears under `--help`
- The result is a namespace whose attributes are the argument names

demo03
first_parser

SpiralTrain developer courses

Options and Flags

- A name starting with a dash is optional, and gets a default

```
parser.add_argument("--dry-run", action="store_true")
parser.add_argument("-v", "--verbose", action="count", default=0)
parser.add_argument("--format", choices=["dec", "hex"], default="dec")
```

- `action="store_true"` makes a flag, present or absent, no value
- `action="count"` counts repeats, so `-vv` gives `2`
- `choices=` rejects anything outside the list and advertises the options
- `nargs="?"` makes a positional optional, with a default of its own
- A dashed name becomes an underscored attribute : `--dry-run` is `args.dry_run`

demo04
options_and_flags

SpiralTrain developer courses

Mutually Exclusive Options

- **Some options make no sense together, and the parser can say so**

```
loudness = parser.add_mutually_exclusive_group()
loudness.add_argument("-v", "--verbose", action="store_true")
loudness.add_argument("-q", "--quiet", action="store_true")
```

- **Passing both is refused before a single line of your code runs :**
  - The message names the conflict and the process exits with status `2`
- **This is validation you would otherwise write, forget, or get subtly wrong**
- **The group is only about the command line, not about the values**

demo05
mutually_exclusive

SpiralTrain developer courses

Generated Help and Errors

- `--help` is generated from the `add_argument` calls, never written by hand

```
usage: squares [-h] [-v] number

positional arguments:
  number         the number to square

options:
  -h, --help     show this help message and exit
  -v, --verbose  explain the answer
```

- A wrong command line prints a usage line to stderr and exits with status `2`
- Both behaviours are free, and both match what every other tool does
- `prog=`, `description=` and `epilog=` fill in the surrounding text

demo06
help_and_errors

SpiralTrain developer courses

Subcommands

- One program, several verbs, the shape of `git`, `apt` and `systemctl`

```
verbs = parser.add_subparsers(dest="command", required=True)

add = verbs.add_parser("add", help="add a host")
add.add_argument("host")
add.set_defaults(handler=cmd_add)
```

- Each verb is a parser of its own, with its own arguments and its own help
- `dest=` names the attribute holding the chosen verb
- `required=True` refuses a call with no verb at all
- `set_defaults(handler=...)` attaches the function that implements the verb

demo07
subcommands

SpiralTrain developer courses

Path Arguments

- `type=` accepts any callable, and `Path` is one

```
from pathlib import Path

parser.add_argument("logfile", type=Path)
args = parser.parse_args()

print(args.logfile.name, args.logfile.stat().st_size)
```

- The argument arrives as a `Path`, ready to be read, joined or inspected
- A converter of your own can validate at the same time :
  - Return the converted value, or raise `argparse.ArgumentTypeError`
  - The parser turns that into an ordinary usage error, at status `2`

demo08
paths

SpiralTrain developer courses

FileType

- `argparse.FileType` opens the file during parsing, and looks convenient
- Python 3.14 deprecates it, with an unusually direct message :

```
FileType is deprecated. Simply open files after parsing arguments.
```

- The reasons :
  - The file is opened even when a later argument turns out to be wrong
  - Nothing ever closes it
- Take a `Path`, then open it yourself, in a `with` block
- Handle the single dash yourself when you want to read standard input

demo09
filetype

SpiralTrain developer courses

main and the Exit Status

- Give the program one entry point that returns an integer

```
def main(argv: list[str] | None = None) -> int:
    args = build_parser().parse_args(argv)
    ...
    return 0

if __name__ == "__main__":
    raise SystemExit(main())
```

- `argv=None` means "take the real command line", and a list means "take this one"
- That default makes the program callable from a test, or from another program
- `SystemExit(0)` is success, and any other number is not
- The convention : `0` fine, `1` ran but found trouble, `2` unusable

demo10
main_exit_codes

SpiralTrain developer courses

Standard Output and Standard Error

- **Two streams, two purposes :**
  - `stdout` carries the result, the part another program will read
  - `stderr` carries progress, warnings and reasons

```
print(record)                              # data
print("reading log", file=sys.stderr)      # everything else
```

- **Mixing them is what makes a tool unpleasant to use in a pipeline**
- **`sys.stdout.isatty()` is `False` when the output is a pipe or a file :**
  - The usual signal to drop colour, banners and progress bars
- **A reader that stops early raises `BrokenPipeError`, which is normal, not a bug**

demo11
stderr_and_pipes

SpiralTrain developer courses

Inline Script Metadata

- PEP 723 lets a single file declare what it needs, in a comment block

```
# /// script
# requires-python = ">=3.12"
# dependencies = ["rich>=13.9"]
# ///
```

- `uv run script.py` reads that block, builds an environment and runs the file
- No virtual environment to create, activate, remember or clean up
- The file stays one file, so it can be mailed, pasted or dropped in `~/bin`
- This is the current answer to "a small program that needs one library"

demo12
pep723_script

SpiralTrain developer courses

Executable Scripts

- Add a shebang above the metadata block and the file becomes a command :

```
#!/usr/bin/env -S uv run --script
```

- Then the usual two steps, and it runs from anywhere :

```
chmod +x logtool.py
./logtool.py count /var/log/syslog
```

- `env -S` is what allows the extra arguments after the interpreter name
- The environment is cached, so only the first run pays for the install
- Pin the versions in `dependencies` and the script still works next year

SpiralTrain developer courses

Command-line Checklist

- It accepts its input on the command line, with sensible defaults
- It has `--help`, and the help is generated rather than maintained
- It refuses a wrong command line clearly, and exits `2` when it does
- It prints data on stdout and everything else on stderr
- It exits `0` on success and non-zero when the answer is bad news
- It reads standard input when the filename is a single dash
- It has one `main(argv) -> int`, so it can be called as well as run

demo13
unix_citizen

SpiralTrain developer courses

Summary : Command-line Programs

- `sys.argv` is the raw list of strings the shell handed over
- `argparse` turns a description of the command line into a parser :
  - Positionals, options, flags, `type=`, `choices=`, `default=` and `help=`
  - The `--help` text and the usage errors come with it
- `add_subparsers` gives one program several verbs, each with its own arguments
- Take a `Path`, then open the file yourself inside a `with` block
- `main(argv: list[str] | None = None) -> int` plus `SystemExit` gives real exit codes
- Data goes to stdout, diagnostics go to stderr, and a broken pipe is normal
- PEP 723 metadata plus `uv run` makes one file a complete program

- Inheritance
- Command-line Programs
- **Modern Toolbelt**
- Web Scraping
- SQLAlchemy
- NumPy
- Pandas
- GUI Toolkits

APPENDIX

- Running Other Programs

- Useful Libraries
- tqdm Progress Bars
- Nested Bars
- Manual Bar Updates
- tqdm.write
- rich Console
- rich Tables
- rich Progress
- rich Tracebacks
- rich inspect
- rich Logging Handler
- HTTP with httpx
- requests and httpx
- typer
- typer Trade-offs
- Finding Libraries
- Judging a Library

SpiralTrain developer courses

Useful Libraries

- **The standard library will do the job, but it will not do it nicely :**
  - A long loop with no output looks identical to a hung program
  - Columns of numbers lined up by hand stop lining up the moment one grows
  - A traceback tells you what broke, but not what the data was
- **Four small libraries fix all of that, and none of them changes how you think :**
  - `tqdm` puts a progress bar around a loop you already wrote
  - `rich` prints colour, tables and readable errors
  - `httpx` fetches things off the web
  - `typer` builds a command line from a function signature
- **None of them is required, and all of them are one `uv add` away**

SpiralTrain developer courses

tqdm Progress Bars

- `tqdm` takes an iterable and returns the same iterable, with a bar :
  - The loop body does not change, and neither does the result
- It measures the rate and estimates the time remaining as it goes

```
from tqdm import tqdm

for server in tqdm(servers, desc="Checking", unit="host"):
    check(server)
```

- `desc` labels the bar, `unit` names the thing being counted
- It needs a length to show a percentage :
  - A list has one, a generator does not, so pass `total=` yourself

demo01
tqdm_basics

SpiralTrain developer courses

Nested Bars

- **Nested loops give nested bars, one line each :**
  - The outer bar counts sites, the inner bar counts hosts within a site
- **`leave=False` on the inner bar is the part people forget :**
  - Without it every finished inner bar stays on screen forever

```
for site, hosts in tqdm(sites.items(), desc="Sites"):
    for host in tqdm(hosts, desc=f"  {site}", leave=False):
        check(host)
```

- **The outer bar's estimate gets better with every site it finishes**

demo02
tqdm_nested

SpiralTrain developer courses

Manual Bar Updates

- **Not all work is a loop over items, so `tqdm` can be driven by hand :**
  - Create it with a `total`, then call `update()` with how much you did
- **Use it as a context manager so the bar is closed properly on the way out**

```
with tqdm(total=size, desc="Reading", unit="B", unit_scale=True) as bar:
    while chunk := handle.read(65536):
        process(chunk)
        bar.update(len(chunk))
```

- **`unit_scale=True` turns 1500 into 1.50k, which keeps the width steady**
- **`bar.set_postfix(ok=12, failed=2)` hangs a live tally off the right-hand end**

demo03
tqdm_manual

SpiralTrain developer courses

tqdm.write

- A bar owns one line and repaints it by writing over itself
- A plain `print()` writes a newline into the middle of that :
  - The bar loses the line it was drawing on
  - Every repaint leaves another half-finished copy behind
- `tqdm.write()` clears the line, prints, and repaints the bar underneath

```
for server in tqdm(servers, desc="Scanning"):
    if is_full(server):
        tqdm.write(f"{server['hostname']} is nearly full")
```

- Same arguments as `print`, so converting a script is a search and replace
- Once a bar exists, nothing in the program may call `print` again

demo04
tqdm_write

SpiralTrain developer courses

rich Console

- `rich` replaces `print` with a `Console` that knows what it is printing :
  - Strings get markup, everything else gets pretty-printed and coloured
- Markup is square brackets, closed with a bare `[/]`, and it nests

```
from rich.console import Console

console = Console()
console.print("[bold]disk usage [red]91%[/] on [blue underline]/srv[/][/]")
console.print({"host": "db01", "cores": 16, "roles": ["primary"]})
```

- It detects that output is being redirected and drops the colour by itself
- `Console(stderr=True)` writes to standard error, so pipelines still work
- `console.rule("Title")` draws a full-width divider, useful on a projector

demo05
rich_console

SpiralTrain developer courses

rich Tables

- A `Table` is declared column by column, then fed one row at a time :
  - Alignment, width and style belong to the column, not to the row
- The widths are computed from the data, so nothing has to be guessed

```
table = Table(title="Server inventory", header_style="bold magenta")
table.add_column("Host", style="cyan", no_wrap=True)
table.add_column("Used", justify="right")
table.add_row("db01", "[red]91%[/]")
console.print(table)
```

- Per-cell colour goes in the text as markup, not as an argument
- `row_styles=["", "dim"]` gives zebra striping for free
- `show_footer` plus `column.footer` produces a totals row

demo06
rich_table

SpiralTrain developer courses

rich Progress

- `track()` is the one-line form, and behaves like `tqdm` :

```
from rich.progress import track

for host in track(hosts, description="Collecting facts"):
    collect(host)
```

- `Progress` is the version for when one bar is not enough :
  - Several tasks on screen at once, each with its own total
  - The bar is assembled from column objects rather than a format string
- Inside a `Progress` block, `print` is redirected and cannot corrupt the display
- This is the one place `rich` is meaningfully easier than `tqdm`

demo07
rich_progress

SpiralTrain developer courses

rich Tracebacks

- **One line at the top of a program restyles every uncaught exception :**

```
from rich.traceback import install

install(show_locals=True)
```

- **The frames get syntax highlighting and the surrounding lines of source**
- **`show_locals=True` is the reason to bother :**
  - The standard traceback says a `ValueError` happened
  - This one shows the row of data that caused it, with no added `print`
- **`console.print_exception()` does the same inside an `except` block**

demo08
rich_traceback

SpiralTrain developer courses

rich inspect

- The usual way to explore an unfamiliar object is `dir()` and a lot of squinting
- `rich.inspect` prints the same information as a readable panel :
  - The repr at the top, then every attribute with its current value

```
from rich import inspect

inspect(connection, methods=True)
```

- `methods=True` adds the call signatures, for "what can I do with this"
- `help=True` adds the docstrings, for "what does this one mean"
- Worth memorising, because it works on anything, including modules

demo09
rich_inspect

SpiralTrain developer courses

rich Logging Handler

- Nothing about the `logging` API changes, only the handler does :

```
import logging
from rich.logging import RichHandler

logging.basicConfig(level="INFO", format="%(message)s",
                    handlers=[RichHandler(rich_tracebacks=True)])
log = logging.getLogger("patchrun")
```

- Levels get colour, times get a column, and the source line is shown
- `rich_tracebacks=True` means `log.exception()` renders the pretty traceback
- Every library that logs through `logging` is improved at the same time
- The format string shrinks to `"%(message)s"` because the handler adds the rest

demo10
rich_logging

SpiralTrain developer courses

HTTP with httpx

- `httpx.Client` sends HTTP requests and reuses connections.
- `MockTransport` supplies a local response for this offline example.

```
import httpx


def example_response(request: httpx.Request) -> httpx.Response:
    return httpx.Response(200, json={"host": "db01", "disk_pct": 73})


transport = httpx.MockTransport(example_response)
with httpx.Client(transport=transport, timeout=10.0) as client:
    response = client.get("https://example.org/hosts/db01")
    response.raise_for_status()
    print(response.json())  # {'host': 'db01', 'disk_pct': 73}
```

- `raise_for_status()` raises an exception for an HTTP error response.
- For a live service, remove `transport=transport` and use its URL.

demo11
httpx_client

SpiralTrain developer courses

requests and httpx

- `requests` :
  - Installed with `uv add requests`
  - Unchanged and stable for a decade
  - Synchronous only
  - No timeout unless you set one
- `httpx` :
  - **Same names, same shapes**
  - **Also does `async` and HTTP/2**
  - **Has a default timeout**
  - **Testable without a network**

- **Either is fine for a synchronous script that fetches a few pages**
- **Choose `httpx` when you need `async`, HTTP/2, or tests that never leave the machine**
- **The scraping module uses `requests`, because that is what the examples use**

demo12
requests_or_httpx

SpiralTrain developer courses

typer

- `typer` builds the parser out of the function it is decorating :
  - The parameter name becomes the option name
  - The annotation becomes the type conversion
  - The default becomes the default, and the docstring becomes the help

```
@app.command()
def show(
    hostname: Annotated[str, typer.Argument(help="The host to describe.")],
    site: Annotated[str | None, typer.Option("--site", "-s")] = None,
) -> None:
    """Show one host in full."""
```

- There is no parser object, no `add_argument`, and no `dest` to spell twice
- `--help` is generated, coloured, and correct without being written

demo13
typer_cli

SpiralTrain developer courses

typer Trade-offs

- **It is a dependency, and `argparse` is not :**
  - A script using `argparse` runs on any Python anywhere, forever
  - A script using `typer` needs an environment, which is what `uv` is for
- **It is a thin layer over the same ideas from module 2 :**
  - Positionals, options, flags, types, defaults, subcommands, exit codes
  - `typer.Exit(code=1)` is how a command reports failure
- **Learn `argparse` first, because it is always there and always readable**
- **Reach for `typer` when a tool grows past a handful of options**

SpiralTrain developer courses

Finding Libraries

- **Almost everything is on PyPI, and the search there is the starting point :**
  - `pypi.org`, then the project's own documentation site
- **Better signals than the search ranking :**
  - What the documentation of a library you already trust recommends
  - What comes up repeatedly in answers to the question you are asking
- **Adding one is a single command, and it records the decision :**

```
uv add rich
```

- **That writes the dependency into `pyproject.toml` and pins it in `uv.lock`**
- **Anyone who runs `uv sync` afterwards gets exactly the versions you had**

SpiralTrain developer courses

Judging a Library

- **Before depending on something, look for five things :**
  - A release within the last year or so, and a visible changelog
  - Real documentation rather than a README with three examples
  - An OSI licence, stated plainly, that your employer can live with
  - Issues that get answered, even if not always fixed
  - A dependency list you would be willing to own
- **Every dependency is code you are trusting and will have to keep updated**
- **Much of this is readable from the metadata already on your disk**

```
import importlib.metadata as metadata

metadata.distribution("rich").version
```

demo14
judging_a_library

SpiralTrain developer courses

Summary : Modern Toolbelt

- Four imports that will make your next script look finished :

```
from tqdm import tqdm
from rich.console import Console
from rich.table import Table
from rich import inspect
```

- A loop that takes more than a second or two should show a bar
- A report with more than two columns should be a `Table`
- An object you do not understand should be handed to `inspect`
- Everything else here is optional, and none of it changes how Python works

- Inheritance
- Command-line Programs
- Modern Toolbelt
- ▶ **Web Scraping**
- SQLAlchemy
- NumPy
- Pandas
- GUI Toolkits

APPENDIX

- Running Other Programs

- Web Scraping
- Fetching a Page
- Status Codes and Timeouts
- BeautifulSoup
- find and find_all
- Text and Attributes
- CSS Selectors
- Navigating the Tree
- Missing Elements
- HTML Tables
- Extracting Records
- SQLite Schema
- Parameterised Inserts
- SQL Injection
- Upsert
- Scrape Pipeline
- robots.txt and Rate Limiting
- Scraping versus an API
- Database Handover

Web Scraping

- **Every scraper is the same three steps, in the same order :**
  - Fetch, parse, store
- **They stay separate on purpose :**
  - Each step can be tested and re-run without the others
  - Only the middle step knows anything about HTML



SpiralTrain developer courses

Fetching a Page

- **`requests.get` returns a response object, not a string :**
  - `.text` is the decoded body, `.content` is the raw bytes
- **Two arguments matter far more than the rest :**
  - `timeout`, because there is no default and a hung server hangs you
  - `headers`, to say which program is asking

```
response = requests.get(
    "https://books.toscrape.com/catalogue/category/books/mystery_3/index.html",
    timeout=10,
    headers={"User-Agent": "MyScraper/1.0 (you@example.com)"},
)
response.raise_for_status()
print(response.status_code, len(response.text))
```

demo01
fetching_pages

SpiralTrain developer courses

Status Codes and Timeouts

- **The status code is the first thing to check :**
  - `200` fine, `301` and `302` redirects that `requests` follows for you
  - `404` gone, `403` refused, `429` slow down, `5xx` their problem
- **`raise_for_status` turns any `4xx` or `5xx` into an exception :**
  - Nothing happens for `2xx` and `3xx`, so it is safe to always call
- **`response.encoding` can be set when a page is decoded wrongly**

```
200 OK                    passes, response.ok is True
404 Not Found             raises HTTPError
429 Too Many Requests     raises HTTPError
```

BeautifulSoup

- **`BeautifulSoup` takes the markup and the name of a parser :**
  - `"lxml"` is fast and forgiving, and is installed for this course
  - `"html.parser"` is built into Python and needs nothing extra
- **The result is a tree you can search, not a string :**
  - Real pages are malformed, and the parser quietly repairs them
- **`response.text` is already decoded, so it goes straight in :**
  - The server declares the encoding, and a price of £51.77 proves it worked

```
from bs4 import BeautifulSoup

html = requests.get(URL, timeout=10).text
soup = BeautifulSoup(html, "lxml")
print(soup.title.text)
```

demo02
making_soup

SpiralTrain developer courses

find and find_all

- **`find` returns the first match, or `None` :**
  - `find_all` returns a list, which may be empty
- **Both take a tag name, and keyword arguments for attributes :**
  - `class_` has the underscore because `class` is a Python keyword
- **Searching a tag searches inside that tag only**

```
pods = soup.find_all("article", class_="product_pod")
for pod in pods:
    title = pod.find("h3").find("a")["title"]
    price = pod.find("p", class_="price_color").text
    print(title, price)
```

demo03
finding_the_books

SpiralTrain developer courses

Text and Attributes

- `.text` collects every string inside a tag :
  - `get_text(strip=True)` is the same thing with the whitespace tidied
- Attributes are read like a dictionary :
  - `tag["href"]` raises `KeyError` when the attribute is absent
  - `tag.get("href")` returns `None`, or a default you supply
- `class` comes back as a list, because an element can carry several

```
stars = pod.find("p", class_="star-rating")
print(stars["class"])              # ['star-rating', 'Four']
print(stars.get("data-value"))     # None, the attribute is not there
```

SpiralTrain developer courses

CSS Selectors

- **`select` and `select_one` take one selector string :**
  - The same language a stylesheet or the browser inspector uses
- **A whole path through the document fits in one argument :**
  - `article.product_pod h3 a` means a inside h3 inside that article
  - `p.star-rating.Four` means both classes on one element

```
titles = soup.select("article.product_pod h3 a")
crumb = soup.select_one("ul.breadcrumb li.active")
images = soup.select("div.image_container > a > img[alt]")
```

demo04
css_selectors

SpiralTrain developer courses

Navigating the Tree

- **Start at something easy to find and move from there :**
  - `.parent` is one step up, `.find_parent("article")` goes up until it matches
  - `.find_next_sibling("p")` skips the whitespace between tags
- **Useful when the thing you want has no class of its own :**
  - A price is easy to find, the module it belongs to is upward from it

```
price = soup.select_one("p.price_color")
pod = price.find_parent("article")
availability = price.find_next_sibling("p")
print(pod.select_one("h3 a")["title"], availability.get_text(strip=True))
```

demo05
walking_the_tree

SpiralTrain developer courses

Missing Elements

- **Real pages are inconsistent, and a scraper meets all of it :**
  - The attribute missing on one row in five hundred
  - The module that has no price because the item is sold out
- **Decide per field what absence means :**
  - `.get("href")` for optional, square brackets for required
  - A required field that is missing should fail loudly, not silently

```
url = link["href"]                          # required, so let it raise
cover = image.get("data-src", "")           # optional, so a default is fine
if pod.find("p", class_="price_color") is None:
    continue                                # no price, no record
```

HTML Tables

- **A table is the easiest markup to scrape, and the most common :**
  - One header row gives the keys, one body row gives the values
- **`zip(headers, cells, strict=True)` refuses to guess :**
  - A short row raises instead of silently shifting every column

```
headers = [cell.get_text(strip=True) for cell in table.select("tr th")]

rows = []
for row in table.select("tbody tr"):
    cells = [cell.get_text(strip=True) for cell in row.find_all("td")]
    rows.append(dict(zip(headers, cells, strict=True)))
```

demo06
parsing_a_table

SpiralTrain developer courses

Extracting Records

- **One function, one page, a list of dictionaries out :**
  - After this line the markup has stopped mattering
- **Convert while parsing, not later :**
  - A price becomes a `float`, a rating becomes an `int`
  - A relative link becomes an absolute one, with `urljoin`

```
soup = BeautifulSoup(html, "lxml")

books = []
for pod in soup.select("article.product_pod"):
    link = pod.select_one("h3 a")
    books.append({
        "title": link["title"],
        "price_gbp": float(re.sub(r"[^\d.]", "", pod.select_one("p.price_color").text)),
        "url": urljoin(URL, link["href"]),
    })
```

demo07
extracting_records

SQLite Schema

- **The same care as any other schema :**
  - One column per field, with a type, and `NOT NULL` where it is true
  - A natural key, `UNIQUE`, so the same page cannot arrive twice
- **`first_seen` and `last_seen` make a re-run informative**

```
CREATE TABLE IF NOT EXISTS books (
    id           INTEGER PRIMARY KEY,
    title        TEXT    NOT NULL,
    price_gbp    REAL    NOT NULL,
    rating       INTEGER NOT NULL,
    url          TEXT    NOT NULL UNIQUE,
    last_seen    TEXT    NOT NULL
);
```

SpiralTrain developer courses

Parameterised Inserts

- **Placeholders keep the values out of the SQL text :**
  - `?` by position, or `:name` by name
- **Named placeholders take a dictionary as it is :**
  - Which is exactly what the parse step produced
- **`executemany` sends the whole list in one call**

```
INSERT = """
INSERT INTO books (title, price_gbp, rating, url, last_seen)
VALUES (:title, :price_gbp, :rating, :url, :seen_at)
"""
conn.executemany(INSERT, [book | {"seen_at": now} for book in books])
conn.commit()
```

SQL Injection

- **The database is not the only thing that reads the string :**
  - A title containing an apostrophe ends the quoted value early
  - Anything after it is executed as SQL
- **One book on the page proves it :**

```
title = "The Bachelor Girl's Guide to Murder"

f"SELECT ... WHERE title = '{title}'"   sqlite3.OperationalError: near "s"
"SELECT ... WHERE title = ?", (title,)  found 1 row
```

- **The parameterised form is also faster, because the query is reused**

SpiralTrain developer courses

Upsert

- **A scrape that only works the first time is not finished :**
  - The job will be re-run, after a crash, or every night
- **`INSERT ... ON CONFLICT DO UPDATE` is one statement for both cases :**
  - `excluded` is the row that was about to be inserted
  - Columns left out of the `SET` keep the value they had

```
INSERT INTO books (title, price_gbp, url, first_seen, last_seen)
VALUES (:title, :price_gbp, :url, :seen_at, :seen_at)
ON CONFLICT(url) DO UPDATE SET
    price_gbp = excluded.price_gbp,
    last_seen = excluded.last_seen
```

demo09
upsert

SpiralTrain developer courses

Scrape Pipeline

- **One script, read top to bottom, in four labelled sections :**
  - Fetch the page, parse the records, store them, report on the table
- **Nothing is wrapped in a function it does not need :**
  - The whole file is the pipeline, and the order on screen is the order it runs
- **Running it a second time changes `last_seen` and nothing else**

```
uv run demos/04-web-scraping/10_scrape_pipeline.py
```

demo10
scrape_pipeline

SpiralTrain developer courses

robots.txt and Rate Limiting

- `robots.txt` describes which paths a crawler may request.
- `RobotFileParser` reads the rules; this example uses saved text.

```
from urllib.robotparser import RobotFileParser

ROBOTS = "User-agent: *\nDisallow: /basket/\nCrawl-delay: 5\n"
AGENT = "TrainingDemo/1.0"
url = "https://books.toscrape.com/basket/"
rules = RobotFileParser()
rules.parse(ROBOTS.splitlines())
print(rules.can_fetch(AGENT, url))  # False
print(rules.crawl_delay(AGENT))  # 5
```

- Identify the scraper with a `User-Agent` and contact details.
- Respect the site's rules, pause between requests and reuse cached pages.

demo11
being_a_good_citizen

SpiralTrain developer courses

Scraping versus an API

- **Scraping is the last resort, not the first :**
  - An API returns JSON that does not change when the design does
  - A CSV or data export is better still, and often already there
- **Signs it is time to stop :**
  - The page builds itself with JavaScript, so the markup is empty
  - There is a login, a CAPTCHA, or a rate limit aimed at you
  - The markup changes weekly and the parser breaks weekly
- **Look for `/api/`, an RSS feed, or a documented download first**

SpiralTrain developer courses

Database Handover

- **The database is the handover point :**
  - Pandas reads it back with `read_sql`
  - A GUI can put a window on the same table
- **A checked in `.sql` file rebuilds it without the scraper :**
  - So the later blocks work even if nothing was ever scraped

```
seed = Path("data/books-seed.sql").read_text(encoding="utf-8")
conn.executescript(seed)
conn.execute("SELECT title, price_gbp FROM books ORDER BY price_gbp DESC")
```

demo12
rebuilding_from_seed

SpiralTrain developer courses

Summary : Web Scraping

- Fetch, parse and store live in separate functions
- Every request has a timeout and a `User-Agent`
- Parsing is a separate step, so it can be re-run without re-fetching
- Values are converted at the parse step, not at the database
- Inserts are parameterised, and the re-run is an upsert
- The pace respects `robots.txt`, and the log says who you are

- Inheritance
- Command-line Programs
- Modern Toolbelt
- Web Scraping
- ▶ **SQLAlchemy**
- NumPy
- Pandas
- GUI Toolkits

- SQLAlchemy
- sqlite3 and SQLAlchemy
- Engine and URL
- begin and connect
- Bound Parameters
- Table Definitions
- Core Queries
- Declarative Models
- Session
- ORM Queries
- Relationships
- Joins and Relationships
- Reflection
- SQLAlchemy and Pandas
- text, Core and ORM

APPENDIX

- Running Other Programs

SQLAlchemy

- **The scraping module talked to SQLite with `sqlite3` :**
  - Which is the driver, and drivers are database-specific
- **SQLAlchemy sits above the driver :**
  - One way of working, whichever database is underneath
- **It has two halves, and both are useful :**
  - `Core`, which builds SQL out of Python objects
  - The `ORM`, which maps rows onto classes
- **Nothing here replaces knowing SQL :**
  - It repays knowing SQL, which is the point

SpiralTrain developer courses

sqlite3 and SQLAlchemy

- **The same call, against a different database :**
  - Change the URL, not the program
- **Parameters have names instead of positions**
- **Rows can come back as objects rather than tuples**
- **The query is a value you can build up, print and reuse**

```
engine = create_engine("sqlite:///:memory:")
with engine.begin() as connection:
    found = connection.execute(
        text("SELECT title FROM book WHERE price > :floor"), {"floor": 25}
    )
```

SpiralTrain developer courses

Engine and URL

- **An engine is a factory and a connection pool :**
  - Make one per database, once, and keep it
  - It does not open a connection until something asks
- **The URL says everything about where the data lives :**

```
dialect+driver://user:password@host/database

sqlite:///:memory:
sqlite:///data/library.db
postgresql+psycopg://feiko@localhost/inventory
```

- **SQLite needs no server, so the middle of the URL is empty**

demo02
engine_and_connection

SpiralTrain developer courses

begin and connect

- `engine.begin()` **opens a transaction :**
  - Commits when the module ends, rolls back if it raises
- `engine.connect()` **gives you a connection and nothing else :**
  - Anything written is discarded unless you commit it yourself
- **Prefer** `begin()` **for anything that writes**

```
with engine.begin() as connection:
    connection.execute(text("INSERT INTO note VALUES ('kept')"))

with engine.connect() as connection:
    rows = connection.execute(text("SELECT body FROM note")).scalars().all()
```

SpiralTrain developer courses

Bound Parameters

- **The parameter is sent separately from the statement :**
  - The database never parses the value as SQL
- **Named, so there are no question marks to count**
- **The same lesson as the scraping module, one layer up**

```
statement = text("SELECT title FROM book WHERE rating >= :floor")
connection.execute(statement, {"floor": 3})
```

- **A formatted query breaks on ordinary data long before it is attacked :**

```
SELECT rating FROM book WHERE title = 'The Widow's Tale'
```

demo03
parameters

SpiralTrain developer courses

Table Definitions

- **Core describes a table as a Python object :**
  - Columns, types, constraints, all as values
- **`MetaData` is the catalogue every table registers in**
- **`create_all()` issues `CREATE TABLE` for whatever is missing**

```
book = Table(
    "book",
    metadata,
    Column("id", Integer, primary_key=True),
    Column("title", String(200), nullable=False),
    Column("price_gbp", Float, nullable=False),
)
```

demo04
describing_a_table

SpiralTrain developer courses

Core Queries

- `select()` returns something you can hold, print and add to :
  - `.where()`, `.order_by()`, `.limit()` each return a new query
- Column access is `table.c.name`
- Printing the query shows the SQL it will send

```
statement = select(book.c.title, book.c.price_gbp).where(book.c.rating >= 4)
print(statement)

with engine.connect() as connection:
    for title, price in connection.execute(statement):
        print(title, price)
```

demo05
core_insert_and_select

SpiralTrain developer courses

Declarative Models

- **The ORM maps a class onto a table :**
  - One class, one table, one instance per row
- **Every model inherits from a `DeclarativeBase` :**
  - Which is block one doing real work
- **`Mapped[int]` is an ordinary annotation, so editors understand it**

```
class Book(Base):
    __tablename__ = "book"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    title: Mapped[str] = mapped_column(String(200))
    price_gbp: Mapped[float] = mapped_column(Float)
```

demo06
declarative_model

Session

- **A `Session` is a unit of work, not a connection :**
  - Add objects to it, then commit once
- **Ids appear after the commit, because the database assigns them**
- **The session tracks what it loaded :**
  - Change an attribute and there is no `UPDATE` to write

```
with Session(engine) as session:
    session.add_all([first, second])
    session.commit()

    first.price_gbp = 41.99
    session.commit()
```

demo07
session_and_persistence

SpiralTrain developer courses

ORM Queries

- The same `select()`, but the rows are objects
- `scalars()` unwraps a one-column result into the thing itself
- Comparisons build SQL rather than returning a boolean :
  - `Book.rating >= 4` is a fragment of a `WHERE` clause

```
found = session.scalars(select(Book).where(Book.rating >= 4)).all()

by_rating = select(Book.rating, func.count(), func.avg(Book.price_gbp)).group_by(
    Book.rating
)
```

demo08
orm_queries

SpiralTrain developer courses

Relationships

- **A foreign key becomes an attribute on both sides :**
  - `category.books` is a list, `book.category` is an object
- **`back_populates` names the attribute pointing the other way**
- **Appending to the list is what creates the link :**
  - No ids written by hand, and the insert order is worked out

```
class Category(Base):
    books: Mapped[list[Book]] = relationship(back_populates="category")

mystery.books.append(Book(title="Sharp Objects"))
session.add(mystery)
```

Joins and Relationships

- **As a join :**
  - One query
  - Explicit, and familiar
  - What the database does anyway

```
select(Book.title, Category.name).join(
    Book.category
)
```

- **As an attribute :**
  - **Reads better**
  - **Issues a query per object**
  - **Easy to do by accident**

```
for book in session.scalars(select(Book)):
    print(book.category.name)
```

demo10
joins

Reflection

- **Reflection reads the schema out of an existing database :**
  - Rather than describing it in Python first
- **Which is most databases, most of the time**

```
metadata = MetaData()
metadata.reflect(bind=engine)

books = metadata.tables["books"]
print([column.name for column in books.columns])
```

- **The demo points at the database the scraping module filled in**

demo11
reading_the_scraped_db

SpiralTrain developer courses

SQLAlchemy and Pandas

- **`read_sql` takes an engine anywhere it takes a connection :**
  - So the portable half comes along for free
- **The grouping can happen in either place :**
  - In the database, or in the DataFrame afterwards
- **For twenty rows it makes no difference :**
  - For twenty million, let the database do it

```
engine = create_engine(f"sqlite:///{DB}")
frame = pd.read_sql("SELECT title, price_gbp, rating FROM books", engine)
```

demo12
with_pandas

SpiralTrain developer courses

text, Core and ORM

- `text()` when the SQL is the clearest way to say it :
  - Or when it is already written and correct
- `Core` when the query is assembled from parts, or must be portable
- The `ORM` when rows are things the program keeps hold of and changes
- These are not levels, and not a progression :
  - Plenty of good programs use all three in one file

demo13
core_or_orm

SpiralTrain developer courses

Summary : SQLAlchemy

- SQLAlchemy sits above the driver, not above SQL
- An engine is made once and carries the URL :
  - Use `begin()` for anything that writes
- Core builds queries as objects you can print and compose
- The ORM maps a class onto a table, with a `Session` as the unit of work
- Relationships turn a foreign key into an attribute on both sides
- Reflection reads a schema somebody else designed
- An engine drops straight into `pandas.read_sql`

- Inheritance
- Command-line Programs
- Modern Toolbelt
- Web Scraping
- SQLAlchemy
- ▶ **NumPy**
- Pandas
- GUI Toolkits

APPENDIX

- Running Other Programs

- NumPy Arrays
- Memory Use
- Speed
- Creating Arrays
- Array Data Types
- Shape and Dimensions
- Reshape and Transpose
- Indexing and Slicing
- Views and Copies
- Vectorised Arithmetic
- Boolean Masking
- np.where
- Broadcasting
- Aggregations
- axis Argument
- Random Numbers
- Reading a CSV

SpiralTrain developer courses

NumPy Arrays

- **NumPy adds one thing to Python, the array :**
  - A fixed-size block of numbers, all of the same type
- **Everything else in the library exists to build or work on that array :**
  - Arithmetic, statistics, sorting, reshaping, reading and writing
- **An operation applies to the whole array at once :**
  - No loop is written, and no loop runs in Python
- **It is the foundation the rest of the numeric stack sits on :**
  - Pandas, SciPy, scikit-learn and matplotlib all speak arrays
- **The usual import, and it is always spelled this way :**

```
import numpy as np

celsius = np.array([25.3, 24.8, 26.9, 23.9])
print(celsius * 9 / 5 + 32)          # every reading converted
```

demo01
first_array

SpiralTrain developer courses

Memory Use

- **A Python list of numbers is a list of pointers :**
  - Each pointer leads to a separate integer object with its own header
- **A NumPy array is one contiguous block of raw numbers :**
  - No per-element object, no per-element pointer
- **The difference is not marginal :**

```
1,000,000 integers
  as a Python list :   36,000,056 bytes
  as a NumPy array :    8,000,000 bytes
```

- **One Python integer costs 28 bytes, one array element costs 8**
- **Contiguous also means the processor can prefetch it :**
  - Which is the reason the next slide comes out the way it does

demo02
memory_footprint

SpiralTrain developer courses

Speed

- Multiply two sequences together and total the result, the Python way :

```
total = 0.0
for i in range(COUNT):
    total += left_list[i] * right_list[i]
```

- The same arithmetic with arrays is `float((left * right).sum())`
- Two million multiplications, timed :

```
Python loop      0.086 s
NumPy arrays     0.008 s
```

- The loop still happens, in compiled C rather than in Python

demo03
speed_comparison

SpiralTrain developer courses

Creating Arrays

- **From data you already have :**
  - `np.array([3, 1, 4])` , `np.array([[1, 2], [3, 4]])`
- **From a rule, when the values are regular :**
  - `np.arange(0, 10, 2)` steps like `range` , and stops one short
  - `np.linspace(0.0, 1.0, 5)` asks for a count, and includes both ends
- **From a shape, when the values come later :**
  - `np.zeros((2, 4))` , `np.ones(3, dtype=int)` , `np.full(4, 7.5)`
- **A shape is a tuple, so a two-dimensional shape needs the inner brackets :**

```
np.zeros(3)             # shape (3,)
np.zeros((3, 1))        # shape (3, 1), a column
```

demo04
making_arrays

SpiralTrain developer courses

Array Data Types

- **Every array has a single element type, its `dtype` :**
  - Chosen once when the array is created, and fixed for its lifetime
- **NumPy picks the narrowest type that holds everything given :**

```
np.array([1, 2, 3]).dtype         # int64
np.array([1.0, 2, 3]).dtype       # float64, one float promotes the lot
```

- **Ask for a type and it is enforced, not negotiated :**

```
np.array([1.7, 2.9, -3.4], dtype=np.int32)   # [ 1  2 -3], truncated
```

- **The common types are `int32` , `int64` , `float32` , `float64` and `bool`**
- **`astype` converts, and always makes a new array**

demo05
dtypes

SpiralTrain developer courses

Shape and Dimensions

- **`shape` is a tuple giving the size along each dimension :**
  - `ndim` is how many dimensions, `size` is the total element count
- **Shape reads outermost first, so a table is rows then columns :**

```
table = np.array([[3.4, 8.7, 9.9],
                  [1.1, -7.8, -0.7]])
print(table.ndim)      # 2
print(table.shape)     # (2, 3), two rows of three columns
print(table.size)      # 6, the product of the shape
```

- **One dimension is a row of values, two is a table, three is a stack of tables**
- **Almost every NumPy error message is about a shape :**
  - Reading the two shapes in the message is usually the whole diagnosis

demo06
shape_and_ndim

SpiralTrain developer courses

Reshape and Transpose

- `reshape` gives the same numbers a different shape :
  - The element count has to come out the same
- `-1` means work the remaining dimension out for me :

```
readings = np.arange(12)
table = readings.reshape(3, 4)      # 3 rows of 4
print(readings.reshape(-1, 2).shape)  # (6, 2)
```

- `.T` transposes, turning rows into columns :

```
print(table.T.shape)                # (4, 3)
```

- Neither call moves a single number :
  - Both hand back another way of looking at the same block of memory

SpiralTrain developer courses

Indexing and Slicing

- **Indexing works as it does on a list :**
  - `row[0]`, `row[-1]`, `row[2:5]`, `row[::2]`, and the stop is excluded
- **In two dimensions the indices go inside one pair of brackets :**

```
table[1, 2]         # row 1, column 2, a single value
table[0]            # a whole row
table[:, 1]         # a whole column, the colon means "all of them"
table[1:, 2:]       # a sub-table, from row 1 and column 2 on
```

- **A slice can also go on the left of an assignment :**

```
table[0] = 0        # the whole first row, in one statement
```

- **The colon means every position along that dimension**

demo08
indexing_and_slicing

SpiralTrain developer courses

Views and Copies

- **Slicing a list gives a copy, slicing an array gives a view :**
  - A view is a window onto the original numbers, not a duplicate of them

```
readings = np.arange(10) * 10
window = readings[1:4]
window[0] = 999
print(readings)     # [0 999 20 30 ...], the original changed
```

- **`.copy()` is the explicit way to detach, and the only reliable one :**

```
window = readings[1:4].copy()      # now edits stay local
```

- **Two ways to check which one is in your hands :**
  - `np.shares_memory(window, readings)` and `window.base`
- **Indexing with a list of positions copies, slicing does not**

demo09
views_and_copies

SpiralTrain developer courses

Vectorised Arithmetic

- **An operator between an array and a number applies to every element :**

```
traffic = np.array([1_200, 3_400, 900, 15_800])
print(traffic / 1024)          # every value in kilobytes
```

- **An operator between two arrays of the same shape works elementwise :**

```
limits = np.array([2_000, 2_000, 2_000, 10_000])
print(limits - traffic)        # headroom, per interface
```

- **The named functions behave the same way over the whole array :**
  - `np.sqrt`, `np.log10`, `np.abs`, `np.round`, and the rest
- **A comparison is arithmetic too, and returns an array of `True` and `False`**
- **This is the habit to build : say what to compute, not how to iterate**

demo10
vectorised_arithmetic

Boolean Masking

- A comparison over an array gives a mask, which is itself an array :

```
latency = np.array([12.1, 9.8, 210.5, 11.2, 480.0])
mask = latency > 100.0          # [False False True False True]
```

- Indexing with the mask keeps the elements where it is `True` :

```
print(latency[mask])            # [210.5 480. ]
print(mask.sum())               # 2, because True counts as one
```

- Conditions combine with `&` and `|`, and each one needs its own brackets :

```
(latency > 12.0) & (latency < 100.0)
```

- The mask also works on the left of an assignment

demo11
boolean_masking

SpiralTrain developer courses

np.where

- `np.where` is an if-else evaluated for every element at once :

```
disk = np.array([41, 88, 96, 12, 73, 91])
print(np.where(disk > 90, 0, 90 - disk))    # headroom, floored at zero
print(np.where(disk > 90, "FULL", "ok"))    # a label per reading
```

- Either branch can be an array or a single value, in any combination
- Nesting gives more than two outcomes, and three is the readable limit :

```
np.where(disk > 90, "critical", np.where(disk > 70, "warning", "fine"))
```

- With a single argument it answers "where", giving positions :
  - `np.where(disk > 90)` and `np.nonzero(disk > 90)` are the same call
- `np.clip(disk, 0, 90)` is the common case, already named

demo12
np_where

SpiralTrain developer courses

Broadcasting

- **Broadcasting lets arrays of different shapes work together :**
  - Line the shapes up from the right, then compare them
- **Two dimensions are compatible when they are equal, or when one is `1` :**
  - A size-one dimension is stretched, and nothing is copied to do it

```
(3, 4)  with  (4,)     ->  (3, 4)   one row applied to every row
(3, 1)  with  (4,)     ->  (3, 4)   a column against a row
(3, 4)  with  (3,)     ->  refused, nothing lines up
```

- **Reshaping to `(3, 1)` is how a per-row value is applied to whole rows :**

```
load - baseline.reshape(3, 1)      # each server minus its own baseline
```

demo13
broadcasting

Aggregations

- **A reduction turns a whole array into one number :**

```
print(latency.sum(), latency.mean())
print(latency.min(), latency.max(), latency.std())
```

- **`argmin` and `argmax` give the position rather than the value :**
  - Which probe, which hour, which server, and usually the useful answer
- **Boolean reductions answer yes or no across the array :**
  - `mask.any()`, `mask.all()`, and `mask.sum()` for the count
- **A mean is a poor summary of anything skewed :**
  - `np.median` and `np.percentile(latency, 95)` say more, and lie less
- **Every reduction exists as a method and as a function, either is fine**

demo14
aggregations

axis Argument

- Every reduction takes an `axis`, and it names the dimension that disappears
- With no `axis` at all, every dimension goes and one number comes back :

```
LOAD = [[  1,   2,   3,   4],      axis 0 runs down the rows
        [ 10,  20,  30,  40],      axis 1 runs along the columns
        [100, 200, 300, 400]]      shape (3, 4)

LOAD.sum()         ->  1110                  shape ()
LOAD.sum(axis=0)   ->  [111 222 333 444]     shape (4,), one per column
LOAD.sum(axis=1)   ->  [  10  100 1000]      shape (3,), one per row
```

- `axis=0` removes the rows, so a per-column answer survives
- `keepdims=True` leaves the collapsed dimension as size one, ready to broadcast

demo15
the_axis_argument

SpiralTrain developer courses

Random Numbers

- **Make a generator, then ask it for what is needed :**

```
rng = np.random.default_rng(seed=42)

print(rng.random(4))                  # uniform, 0 to 1
print(rng.integers(1, 7, size=8))     # eight dice
print(rng.normal(100.0, 15.0, 4))     # mean 100, deviation 15
```

- **Every call takes a `size`, and a shape works as well as a count**
- **`rng.choice` samples from data already in hand, with or without replacement**
- **A seed makes the run repeatable, which is what a demo or a test needs**
- **The older `np.random.seed` and `np.random.rand` still exist :**
  - They share one hidden global generator, and are not what to write now

demo16
random_numbers

SpiralTrain developer courses

Reading a CSV

- `np.loadtxt` reads a rectangle of numbers, and wants it to be tidy :

```
load = np.loadtxt(CSV, delimiter=",", skiprows=1, usecols=(1, 2, 3, 4, 5))
print(load.shape)                  # (24, 5), hours by servers
print(load.mean(axis=0))           # mean per server
```

- `skiprows` steps over a header, `usecols` drops columns that are not numbers
- `np.savetxt` writes one back out, and `np.save` stores the exact bytes
- `np.genfromtxt` copes with gaps and missing values, at some cost in speed
- When the columns have names and different types, this stops being the tool :
  - That is a table rather than an array, and the next module reads it properly

demo17
csv_into_array

SpiralTrain developer courses

Summary : NumPy

- **Reach for it when the data is a rectangle of numbers of one type :**
  - Measurements, samples, pixels, matrices, counters over time
- **Three habits to carry forward :**
  - Say what to compute, rather than how to loop over it
  - Print `.shape` whenever something is not behaving
  - Write `.copy()` when a slice is about to be modified
- **Reach for Pandas instead when the columns have names and types**
- **The rest of the numeric stack is built on this one object :**
  - SciPy, scikit-learn, matplotlib and Pandas all take arrays and give them back

- Inheritance
- Command-line Programs
- Modern Toolbelt
- Web Scraping
- SQLAlchemy
- NumPy
- **Pandas**
- GUI Toolkits

APPENDIX

- Running Other Programs

- Pandas
- Series
- DataFrame
- The Index
- Reading a CSV
- Reading a Spreadsheet
- head and tail
- info and describe
- Selecting Columns
- loc
- iloc
- loc and iloc
- Filtering Rows
- Combining Filters
- Adding a Column
- Deriving a Column
- Sorting
- groupby
- groupby Aggregation
- Named Aggregation
- merge
- Join Types
- Missing Values
- read_sql
- to_sql
- Writing Files
- Plotting

Pandas

- **`pandas` is the library for tables of data :**
  - Rows and columns, where the columns have names and different types
- **The thing it replaces is a spreadsheet, or a pile of parsing code :**
  - A CSV, a sheet of a workbook, a query result, all become the same object
- **Built on top of NumPy :**
  - Each column is an array, so a column operation runs at array speed
- **The working loop is short :**
  - Read a file, look at it, filter it, summarise it, write it out again
- **Two objects carry the whole library :**
  - `Series`, one column · `DataFrame`, a table of them

```
import pandas as pd
```

SpiralTrain developer courses

Series

- **A `Series` is a one-dimensional array with an index attached :**
  - The index is a label per row, not a row number
- **The two halves are reachable separately :**
  - `.index` gives the labels, `.to_numpy()` gives the values
- **Arithmetic and comparison apply to the whole column at once**

```
free_gb = pd.Series([55.5, 48.8, 550.0], index=["web01", "web02", "db01"])
free_gb["db01"]          # 550.0, looked up by label
free_gb / 1024           # every element, no loop
free_gb > 100            # a Series of True and False
```

demo01
series_basics

SpiralTrain developer courses

DataFrame

- A `DataFrame` is a dictionary of `Series` that all share one index :
  - Each column has a name and its own type
- Built from a dictionary, a list of dictionaries, or read from a file
- `.shape`, `.columns` and `.index` are the three facts to check first

```
servers = pd.DataFrame({
    "hostname": ["web01", "web02", "db01"],
    "site": ["AMS", "AMS", "AMS"],
    "cpu_cores": [4, 4, 16],
})
servers.shape            # (3, 3), rows by columns
servers["cpu_cores"]     # one column, which is a Series
```

demo02
dataframe_basics

The Index

- **The index is the row's name, and it travels with the row :**
  - Filtering, sorting and joining all keep it
- **`set_index` promotes a column to be the index**

```
by_host = servers.set_index("hostname")
by_host.loc["db01"]      # a row fetched by name
```

- **Operations align on the label, not on the position :**
  - Two Series in different orders still line up correctly

```
quota = pd.Series([500, 2000], index=["web01", "db01"])
used = pd.Series([1450.0, 64.5], index=["db01", "web01"])
quota - used             # matched by name, not by row
```

demo03
the_index

SpiralTrain developer courses

Reading a CSV

- **`read_csv` takes a path and returns a `DataFrame` :**
  - Column types are inferred from the file, not declared
- **The keywords that rescue somebody else's export :**
  - `sep=` a different delimiter · `index_col=` which column is the key
  - `usecols=` only the columns wanted · `na_values=` what counts as empty
  - `skiprows=` junk at the top · `header=None` no header row at all

```
servers = pd.read_csv(DATA / "servers.csv")
print(servers.head(2))

servers = pd.read_csv(DATA / "servers.csv", index_col="hostname")
print(servers.head(2))
```

demo04
read_csv

SpiralTrain developer courses

Reading a Spreadsheet

- **`read_excel` is the same idea for a workbook :**
  - Needs `openpyxl` installed, which the lockfile already pins
- **A workbook has sheets, so one more argument is needed :**
  - `sheet_name="servers"` by name, `0` by position, `None` for all of them

```
pd.ExcelFile("data/servers.xlsx").sheet_names   # ['servers', 'sites']

servers = pd.read_excel("data/servers.xlsx", sheet_name="servers")
every = pd.read_excel("data/servers.xlsx", sheet_name=None)   # a dict
```

- **Reading every sheet gives a dictionary keyed by sheet name**

demo05
read_excel

SpiralTrain developer courses

head and tail

- `head()` and `tail()` show the ends of the table :
  - Five rows by default, or as many as asked for
- They return a `DataFrame`, so they chain and print like any other
- `sample()` is the honest third option :
  - The head of a file is often sorted, and a sorted head is not typical

```
servers.head()           # first five rows
servers.head(2)          # first two
servers.tail(3)          # last three
servers.sample(3, random_state=0)   # three at random, reproducibly
```

demo06
head_and_tail

SpiralTrain developer courses

info and describe

- **`info()` describes the structure :**
  - Row count, column names, type of each, and how many values are present
- **`describe()` summarises the numbers :**
  - Count, mean, standard deviation, minimum, quartiles, maximum

```
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 6   disk_gb       14 non-null     int64
 7   used_gb       13 non-null     float64
 8   last_patched  12 non-null     str
```

- **The non-null counts are the fastest missing-data check there is :**
  - Any column short of the row count has holes in it

demo07
info_and_describe

SpiralTrain developer courses

Selecting Columns

- One name in square brackets gives a `Series` :
  - `servers["memory_gb"]`
- A list of names gives a `DataFrame` , in the order given :
  - `servers[["site", "role", "memory_gb"]]`
- The double brackets are a list inside a selection, not special syntax

```
servers["memory_gb"]                  # a Series
servers[["memory_gb"]]                # a DataFrame with one column
servers[["role", "site", "disk_gb"]]  # three columns, reordered
```

- Selecting with a list is also how a report gets its column order

demo08
selecting_columns

SpiralTrain developer courses

loc

- **`loc[rows, columns]`, both given as labels :**
  - A single label, a list of labels, or a slice of labels

```
servers.loc["db01", "memory_gb"]              # one cell
servers.loc["db01"]                           # one row, as a Series
servers.loc["db01", ["site", "role"]]         # one row, chosen columns
servers.loc[:, ["site", "role"]]              # every row, two columns
servers.loc["web01":"db01", "site":"os"]      # a module
```

- **A label slice includes both ends, unlike every other slice in Python**

demo09
loc_by_label

iloc

- **`iloc[rows, columns]`, both given as integers :**
  - The labels are ignored entirely, as if the table were a list of lists

```
servers.iloc[0, 4]         # first row, fifth column
servers.iloc[:3, :4]       # first three rows, first four columns
servers.iloc[-2:, :4]      # last two rows
servers.iloc[[0, 5, 2], [0, 1]]   # lists of positions, in that order
```

- **An integer slice stops before its end, exactly like a list slice**
- **The same cell, two ways : `loc["db01", "memory_gb"]` is `iloc[2, 4]`**

demo10
iloc_by_position

loc and iloc

`loc`, by label

- Takes the row and column names
- Slices include both ends
- Survives a re-sort of the table
- What you want most of the time

`iloc`, by position

- Takes integers, counting from zero
- Slices stop before the end
- Changes meaning when rows move
- For "the first" and "the last"

- Both take a comma : rows first, then columns
- Plain `[]` is neither : it selects columns on a frame

SpiralTrain developer courses

Filtering Rows

- **A comparison on a column gives one `True` or `False` per row :**
  - That is an ordinary value, and it can be given a name
- **Indexing the frame with that mask keeps only the matching rows**

```
big = servers["memory_gb"] >= 64      # a Series of booleans
servers[big]                          # the rows where it is True

servers[servers["disk_gb"] > 1000]    # usually written in one line
servers[servers["role"] == "web"]     # text compares just as well
```

- **A mask sums to a count, because `True` is one and `False` is zero**

demo11
boolean_filter

SpiralTrain developer courses

Combining Filters

- **`&` for and, `|` for or, `~` for not :**
  - Not the words `and`, `or` and `not`, which want one answer, not a column
- **Every comparison needs its own parentheses, because `&` binds tighter**

```
servers[(servers["cpu_cores"] >= 8) & (servers["disk_gb"] >= 2000)]
servers[(servers["role"] == "database") | (servers["role"] == "build")]
servers[~servers["site"].isin(["AMS", "RTM"])]
```

- **Two shorthands :**
  - `.isin([...])` replaces a chain of ors
  - `.between(low, high)` replaces two comparisons, and includes both ends

demo12
combining_filters

SpiralTrain developer courses

Adding a Column

- **Assigning to a name that does not exist creates the column :**
  - A scalar is broadcast to every row
  - A `Series` is computed element by element, aligned on the index

```
servers["managed"] = True
servers["free_gb"] = servers["disk_gb"] - servers["used_gb"]
servers["used_pct"] = (servers["used_gb"] / servers["disk_gb"] * 100).round(1)
```

- **`assign` does the same but returns a new frame, so it chains**
- **`drop(columns=[...])` removes columns and also returns a new frame**

demo13
adding_columns

SpiralTrain developer courses

Deriving a Column

- **Three ways to compute a column from another :**
  - `.str` gives every string method, applied down the column
  - `.map(dictionary)` translates codes into labels
  - `np.where(condition, yes, no)` is a vectorised if-else

```
servers["distro"] = servers["os"].str.split().str[0]
servers["tier"] = servers["role"].map({"web": "front line", "db": "critical"})
servers["alert"] = np.where(servers["used_pct"] > 75, "FULL", "ok")
```

- **A missing value never compares true, so it lands in the `no` branch :**
  - Which quietly turns "never measured" into "fine"

demo14
deriving_columns

Sorting

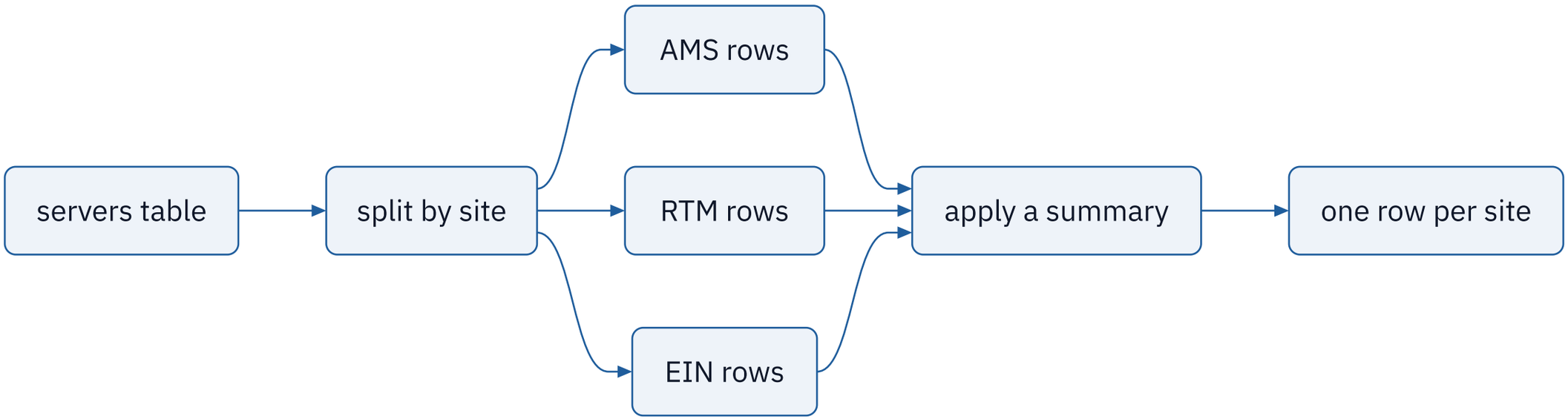
- `sort_values` takes a column name, or a list of them :
  - `ascending=` takes one boolean, or one per column
- It returns a new frame, the original is untouched

```
servers.sort_values("disk_gb", ascending=False)
servers.sort_values(["site", "disk_gb"], ascending=[True, False])
servers.nlargest(3, "used_gb")     # sort plus head, saying what it means
servers.sort_index()               # back into label order
```

- `nlargest` and `nsmallest` read better than a sort followed by a head

demo15
sort_values

groupby



- One row per distinct value of the grouping column comes back

SpiralTrain developer courses

groupby Aggregation

- `groupby("column")`, then a column to summarise, then a summary

```
servers.groupby("site")["cpu_cores"].sum()
servers.groupby("role")["disk_gb"].mean().round(1)
servers.groupby("site").size()                    # how many per group
```

- `agg` takes a list of functions, or a dictionary per column

```
servers.groupby("site")["memory_gb"].agg(["count", "sum", "mean", "max"])
servers.groupby("site").agg({"cpu_cores": "sum", "disk_gb": "max"})
```

- The grouping column becomes the index of the answer

demo16
groupby_agg

SpiralTrain developer courses

Named Aggregation

- **Named aggregation gives each output column the name the report wants :**
  - Keyword is the new column, value is `(column, summary)`
- **`reset_index()` turns the grouping key back into an ordinary column**

```
per_site = (
    servers.groupby("site")
    .agg(servers=("hostname", "count"),
         total_cores=("cpu_cores", "sum"),
         mean_used_gb=("used_gb", "mean"))
    .round(1)
    .reset_index()
)
```

- **Grouping by two keys is the same call with a list of column names**

demo17
groupby_named_agg

SpiralTrain developer courses

merge

- `merge` combines two frames on a shared key column :
  - `on=` names a column that exists in both
  - `left_on=` and `right_on=` when the two sides call it different things

```
servers = pd.read_csv("data/servers.csv")   # has a site code
sites = pd.read_csv("data/sites.csv")       # explains what the code means

joined = servers.merge(sites, on="site")
joined.groupby("city").size()               # only possible after the join
```

- The result is an ordinary frame, so it filters, sorts and groups

demo18
merge_on_key

SpiralTrain developer courses

Join Types

- `how=` decides what happens to rows that match nothing :
  - `inner`, the default, keeps only rows present on both sides
  - `left` keeps every row of the left frame, filling gaps with `NaN`
  - `right` does the mirror image, `outer` keeps everything

```
servers.merge(sites, on="site", how="left")
servers.merge(sites, on="site", how="outer", indicator=True)
```

- `indicator=True` adds a `_merge` column saying where each row came from :
  - `both`, `left_only`, `right_only`
- It answers whether the join lost rows

demo19
merge_how

SpiralTrain developer courses

Missing Values

- An empty field becomes `NaN`, and `NaN` is not equal to anything
- `isna()` gives a mask of the same shape, and a mask sums to a count

```
servers.isna().sum()                          # holes per column
servers[servers.isna().any(axis=1)]           # the rows with any hole
servers.dropna()                              # drop any row with any hole
servers.dropna(subset=["used_gb"])            # only rows missing that column
servers.fillna({"used_gb": 0.0, "last_patched": "never"})
```

- Summaries skip `NaN` silently, so a mean is a mean of what was there

demo20
missing_data

SpiralTrain developer courses

read_sql

- `read_sql` runs a query and returns the result set as a `DataFrame` :
  - Takes any DB-API connection, `sqlite3` included
- Values go in as parameters, never by string formatting

```
import sqlite3

conn = sqlite3.connect("data/books.db")
books = pd.read_sql("SELECT * FROM books", conn)
dear = pd.read_sql("SELECT * FROM books WHERE price_gbp > ?",
                   conn, params=(45.0,))
conn.close()
```

- Push the work the database is good at into the query, then stop

SpiralTrain developer courses

to_sql

- `to_sql` writes a frame into a table, creating it if it is not there :
  - `if_exists=` is `fail` by default, or `replace`, or `append`
  - `index=False` keeps the frame's row labels out of the table

```
conn = sqlite3.connect("data/pandas-demo.db")
servers.to_sql("servers", conn, if_exists="replace", index=False)
conn.commit()
print(pd.read_sql("PRAGMA table_info(servers)", conn))
conn.close()
```

- The round trip is the point : summarise in pandas, store the result back
- Column types are chosen by pandas from the dtypes

demo22
to_sql

SpiralTrain developer courses

Writing Files

- `to_csv` and `to_excel` are the mirror of the readers :
  - `index=False` unless the index is real data somebody wants
- `ExcelWriter` puts several frames into one workbook, a sheet each

```
report.to_csv("data/servers-report.csv", index=False)
report.to_csv("data/servers-report.tsv", sep="\t", index=False)

with pd.ExcelWriter("data/report.xlsx", engine="openpyxl") as workbook:
    report.to_excel(workbook, sheet_name="servers", index=False)
    per_site.to_excel(workbook, sheet_name="per site", index=False)
```

demo23
writing_files

Plotting

- Any frame or column can draw itself with `.plot()` :
  - `kind=` is `line`, `bar`, `barh`, `hist`, `scatter`, `box`
- Choose the non-interactive backend first, and the script writes a file

```
import matplotlib
matplotlib.use("Agg")          # before pyplot, so no window is opened

per_site = servers.groupby("site")[["used_gb", "free_gb"]].sum()
axes = per_site.plot(kind="barh", stacked=True)
axes.get_figure().savefig("data/servers-plot.png", dpi=120)
```

- Good enough to look at, and rarely good enough to publish

demo24
plot_to_file

SpiralTrain developer courses

Summary : Pandas

- `Series` is a column with labels, `DataFrame` is a table of them :
  - The index is a name per row, and everything aligns on it
- Read with `read_csv`, `read_excel`, `read_sql`, write with the `to_` pair
- Look before working : `head`, `tail`, `info`, `describe`
- Address with `loc` by label and `iloc` by position, rows before columns
- Filter with a boolean mask, combined with `&`, `|` and `~`
- Summarise with `groupby().agg()`, combine tables with `merge`
- Handle holes deliberately : `isna`, `dropna(subset=)`, `fillna`

- Inheritance
- Command-line Programs
- Modern Toolbelt
- Web Scraping
- SQLAlchemy
- NumPy
- Pandas
- ▶ **GUI Toolkits**

APPENDIX

- Running Other Programs

- GUI Toolkits
- Installing tkinter
- Root Window and Event Loop
- Widgets
- Widget Values
- grid Layout
- Events and Callbacks
- ttk.Treeview
- Treeview and SQLite
- Choosing a Toolkit
- GUI Trade-offs

GUI Toolkits

- **Everything so far has printed to a terminal :**
  - Which is the right answer more often than people admit
- **Sometimes the audience is not you :**
  - A colleague who will not type a command
  - A tool that has to be handed over and used unattended
- **Python has a graphical toolkit in the standard library :**
  - `tkinter`, which is always present and never has to be installed
- **The goal here is recognition, not a finished application**

Installing tkinter

- **`tkinter` ships with Python, but not with every Python package :**
  - The Windows and macOS installers include it
  - Most Linux distributions split it into a separate package
- **On Debian and Ubuntu it is one command :**

```
sudo apt install python3-tk
```

- **Check it before writing any code :**

```
python -c "import tkinter; print(tkinter.TkVersion)"
```

SpiralTrain developer courses

Root Window and Event Loop

- **A graphical program has a different shape from a script :**
  - A script runs top to bottom and exits
  - A window is created, then waits
- **`Tk()` is the root window, and there is exactly one**
- **`mainloop()` waits for an event, dispatches it, and waits again :**
  - It does not return until the window is destroyed
  - Nothing written after it runs before then

```
root = tk.Tk()
root.title("A window")
root.geometry("420x220")
root.mainloop()
```

demo01
gui_root_window

Widgets

- **Three widgets carry most simple forms :**
  - `Label` shows text, `Entry` takes text, `Button` does something
- **Prefer the themed `ttk` versions over the plain `tk` ones :**
  - Same names, native appearance
- **A widget is created with its parent as the first argument**
- **`command=` takes the function itself, without parentheses**

```
entry = ttk.Entry(root, width=24)
button = ttk.Button(root, text="Look up", command=look_up)
```

demo02
gui_widgets

Widget Values

- **A widget owns its value, and is asked for it rather than tracked :**
  - `entry.get()` reads what the user typed
  - `label.configure(text=...)` writes it back
- **`configure()` sets any option after construction**
- **The callback is an ordinary method :**
  - It runs when the event happens, not when it is registered

```
def look_up() -> None:
    name = entry.get().strip()
    if not name:
        result.configure(text="type a hostname first")
        return
    result.configure(text=f"{name} resolved")
```

SpiralTrain developer courses

grid Layout

- **Widgets do not place themselves, a geometry manager places them**
- **`grid` puts a widget in a row and a column :**
  - `sticky` takes compass points, so `"e"` pins right and `"ew"` fills
  - `padx` and `pady` add space around the widget
  - `columnspan` lets one widget sit across several columns
- **A widget stretches only if its row or column is given a weight :**
  - Weight is a share of the spare space, not a yes or no

```
label.grid(row=row, column=0, sticky="e", padx=(0, 8), pady=4)
entry.grid(row=row, column=1, sticky="ew", pady=4)
form.columnconfigure(1, weight=1)
```

demo03
gui_grid_layout

SpiralTrain developer courses

Events and Callbacks

- **`bind()` maps an event pattern to a callback :**
  - `<Key>` is any key, `<Key-a>` is one key, `<Button-1>` is a left click
  - `<Configure>` fires when the widget is resized
- **Every bound callback is handed one argument, the event :**
  - It carries `x`, `y`, `keysym` and the widget it happened on
- **A `Button`'s `command=` is the same idea with the event thrown away**

```
root.bind("<Key>", lambda event: report(f"key {event.keysym}", event))
canvas.bind("<Button-1>", lambda event: report("left click", event))
canvas.bind("<Configure>", lambda event: report("resized", event))
```

demo04
gui_events

SpiralTrain developer courses

ttk.Treeview

- `ttk.Treeview` is a real table, with headings and columns
- Rows are inserted, and each one gets an identifier back
- Clicking a heading can re-order the rows :
  - `tree.set` reads one cell, `tree.move` re-places a row
- The widget most internal tools need :
  - Most internal tools are a table of something with a filter above it

```
tree = ttk.Treeview(root, columns=list(COLUMNS), show="headings")
for row in rows:
    tree.insert("", "end", values=[row[name] for name in COLUMNS])
```

demo05
gui_treeview

SpiralTrain developer courses

Treeview and SQLite

- **The same table, filled from SQLite instead of a CSV :**
  - Type a filter, press Return, run the query, refill the table
- **The user's text is a parameter, never formatted into the SQL :**
  - Exactly as in the scraping module

```
sql = f"SELECT {', '.join(COLUMNS)} FROM servers"
if term:
    sql += " WHERE site LIKE ? OR role LIKE ? OR hostname LIKE ?"
    parameters = (f"%{term}%",) * 3
return connection.execute(sql, parameters).fetchall()
```

demo06
gui_treeview_sqlite

Choosing a Toolkit

- `tkinter` :
  - In the standard library
  - Plain, dated, acceptable
  - Small tools and dialogs
- `PySide6` , the Qt binding :
  - A large install
  - Native and modern
  - Real desktop applications
- `Streamlit` :
  - **Opens a browser tab**
  - **No event loop, no callbacks**
  - **Dashboards over data**
- **Model and view :**
  - `tkinter` **holds the rows itself**
  - **Qt asks a model for them**
  - **Which matters at scale**

SpiralTrain developer courses

GUI Trade-offs

- **A graphical program costs more than it looks :**
  - It cannot be run from cron, piped, or driven from a script
  - It needs a display, which rules out most servers
  - It is harder to test than anything else in this course
- **Reach for a window when the user is not you :**
  - And reach for a command-line tool when the user is
- **The command-line tools cover most of the middle ground :**
  - A clear `--help`, real exit codes, and a table printed with `rich`

SpiralTrain developer courses

Summary : GUI Toolkits

- **A GUI inverts the shape of a program :**
  - Build the widgets, then hand control to an event loop
- **`tkinter` is in the standard library and always available :**
  - `python3-tk` on Debian and Ubuntu
- **Three widgets and one geometry manager cover most forms :**
  - `Label`, `Entry`, `Button`, placed with `grid`
- **`ttk.Treeview` is the widget most internal tools actually need**
- **Qt and Streamlit are the other two answers, and both are in `reference/gui/`**

- Inheritance
- Command-line Programs
- Modern Toolbelt
- Web Scraping
- SQLAlchemy
- NumPy
- Pandas
- GUI Toolkits

APPENDIX

- ▶ **Running Other Programs**

- Running Other Programs
- subprocess.run
- Capturing Output
- Exit Codes
- Argument Lists
- Standard Input and Pipelines
- Timeouts
- Environment and Working Directory
- Missing Programs
- Popen and Streaming Output
- pathlib Alternatives

SpiralTrain developer courses

Running Other Programs

- **Some work belongs to a program that already exists :**
  - `git` , `systemctl` , `rsync` , a vendor's command-line tool
- **`subprocess` starts it, waits for it, and reports back**
- **This is the module for the script that has outgrown its shell :**
  - The logic got complicated, the quoting got worse
  - The commands themselves were never the problem
- **Everything here is standard library, so there is nothing to install**

SpiralTrain developer courses

subprocess.run

- `run()` starts the program, waits, and returns a `CompletedProcess`
- The command is a list : the program, then each argument separately
- Nothing is captured unless you ask, so output goes straight to the terminal

```
finished = subprocess.run(["git", "--version"], check=False)

print(finished.args)
print(finished.returncode)
```

demo01
running_a_command

SpiralTrain developer courses

Capturing Output

- `capture_output=True` fills in `stdout` and `stderr`
- `text=True` makes them strings rather than bytes :
  - Without it you get `b'18\n'`, which is almost never wanted
- **After that it is ordinary Python, not a shell pipeline**

```
finished = subprocess.run(
    ["wc", "-l", str(log)], capture_output=True, text=True, check=True
)

count = int(finished.stdout.split()[0])
```

demo02
capturing_output

SpiralTrain developer courses

Exit Codes

- **A non-zero exit is information, not always a failure :**
  - `grep` exits 1 when it did not match
- **`check=False` and read `returncode` yourself when the code is meaningful**
- **`check=True` raises `CalledProcessError` when it is not**

```
found = subprocess.run(["grep", "-c", "ERROR", str(log)],
                       capture_output=True, text=True, check=False)
print(found.returncode, found.stdout.strip())
```

demo03
exit_codes

SpiralTrain developer courses

Argument Lists

- **A list hands the arguments to the program directly :**
  - No shell starts, so nothing splits and nothing expands
- **`shell=True` hands the whole string to a shell first :**
  - A space in a filename becomes two arguments
  - Punctuation becomes syntax

```
subprocess.run(["wc", "-l", str(awkward)])          # correct

subprocess.run(f"wc -l {awkward}", shell=True)      # splits on the space
```

demo04
a_list_not_a_string

SpiralTrain developer courses

Standard Input and Pipelines

- `input=` writes to the child's stdin and closes it
- Two commands join without a shell :
  - Run the first, capture it, hand the text to the second
- Clearer than a pipeline string, with nothing for a shell to misread

```
levels = subprocess.run(["grep", "-o", "-E", "INFO|WARN|ERROR", str(log)],
                        capture_output=True, text=True, check=True)
counted = subprocess.run(["uniq", "-c"], input=levels.stdout,
                         capture_output=True, text=True, check=True)
```

demo05
input_and_pipes

SpiralTrain developer courses

Timeouts

- **A child that hangs will hang the parent for ever**
- **`timeout=` kills it and raises `TimeoutExpired` :**
  - Nothing is left running behind you
- **Any command touching a network, a device or another machine wants one**

```
try:
    subprocess.run(slow_command, timeout=1.5, check=True)
except subprocess.TimeoutExpired as error:
    print("gave up after", error.timeout, "seconds")
```

demo06
timeouts

SpiralTrain developer courses

Environment and Working Directory

- **The child inherits this process's environment by default**
- **`env=` replaces it wholesale :**
  - Copy `os.environ` and add to the copy, or the child loses `PATH`
- **`cwd=` runs the command elsewhere without moving yourself :**
  - Better than `os.chdir()`, which changes it for everything

```
environment = os.environ.copy()
environment["GREETING"] = "set for the child only"
subprocess.run(command, env=environment, check=True)
```

demo07
environment

SpiralTrain developer courses

Missing Programs

- A missing command raises `FileNotFoundError`, not a non-zero exit
- `shutil.which()` answers the question without running anything
- Check dependencies at the start of a script, not at the point of use :
  - Failing immediately with a clear message beats failing halfway through

```
if shutil.which("rsync") is None:
    sys.exit("rsync is not installed")
```

demo08
finding_the_program

SpiralTrain developer courses

Popen and Streaming Output

- `run()` returns nothing until the command has finished
- `Popen` starts the process and hands back control immediately :
  - Iterating `stdout` yields lines as the child flushes them
- Use it for anything long enough that silence looks like a hang

```
with subprocess.Popen(command, stdout=subprocess.PIPE,
                      text=True, bufsize=1) as process:
    for line in process.stdout:
        print("received :", line.rstrip())
```

demo09
streaming_output

SpiralTrain developer courses

pathlib Alternatives

- Starting a process costs time and brings quoting with it
- Python already does most of what the small commands do :

```
ls *.log      ->  Path.glob             basename  ->  Path.name
wc -l         ->  str.splitlines        test -f   ->  Path.exists
grep          ->  a comprehension       find      ->  Path.rglob
cp / mv / rm ->  shutil.copy, Path.rename, Path.unlink
```

- Reach for `subprocess` when the work belongs to another program

SpiralTrain developer courses

Summary : Running Other Programs

- `subprocess.run()` starts a program, waits, and reports back
- Pass a list, not a string, and leave `shell=True` alone
- `capture_output=True` with `text=True` to read what it printed
- `check=True` to raise on failure, `check=False` when the code means something
- `timeout=` on anything that could hang
- `shutil.which()` to fail early with a clear message
- `Popen` when you need the output as it arrives
- And `pathlib` instead, for anything the filesystem already does